

Laboratório de Microcontroladores - Relatório da aula prática 7

Semáforo inteligente com *buzzer* passivo

20249035626 - Gabriel de Oliveira Barbosa – Quarta-feira – 2026.1

1. Resumo da prática:

Na prática realizada em sala, tinha como objetivo implementação da função adicional do *buzzer* no semáforo, a montagem é essencialmente a mesma forma do código apresentado no relatório da prática 6, somente adicionado a função do *buzzer*, e suas interações com a função dos casos *switch*, da implementação das funções de *tone* utilizadas para o funcionamento do equipamento sonoro digital, e da definição dos pinos para o funcionamento deste componente.

2. Objetivos:

Adaptar com base nos conhecimentos adquiridos nas montagens anteriores uma montagem para a implementação de um sistema que agora inclua a função de *buzzer* sonoro digital para o reconhecimento do pedido do pedestre, e que coincida com o funcionamento da função de piscar para o semáforo do pedestre.

3. Material Utilizado:

- 3.1 |1x Arduino Nano
- 3.2 |Cabo MiniUSB
- 3.3|1x Protoboards.
- 3.4 |Cabos Jumper macho-macho.
- 3.5 |5x Leds.
- 3.6 |1x Buzzer.
- 3.7 | 1x Botão push.
- 3.8 |Computador/Notebook.
- 3.9 |1x Resistor 10Kohms.
- 3.10 |5x Resistor 220ohms.

4. Montagem:

A montagem seguiu os procedimentos orientados no documento da prática 7, disponibilizado no grupo da sala, com a placa Nano fornecida, foi organizado os cabos de acordo com os pinos definidos no código, e para fim de economia de cabos utilizados, foi usado a extremidade do cátodo dos leds diretamente ligada ao aterramento da placa. Foi usado o resistor de 10kohms no botão *push*, os 6 outros resistores de 220ohms foram usados nos 5 leds, 2 de cor verde, 2 de vermelho e 1 de amarelo. Já o último resistor 220 ohms foi utilizado para a ligação da alimentação e transporte de informações para o *buzzer digital*, que ligado pela porta 6, e conectada pelo fio jumper e o resistor, consegue estabelecer sua comunicação. Após montado, foi importante verificar se havia algum tipo de curto na corrente dos componentes, e verificado no terminal IDE, se estava configurado de acordo com os conformes estabelecidos para a execução do projeto. O resultado deve ser algo como observado na figura 5 no tópico 7. Também é usada no código a função *tone*, que define o pino, frequência e a duração do bipe para o *buzzer* sonoro (este conectado ao pino 6)

5. Resultados da montagem:

Após corrigido o código utilizado da montagem 6, implementamos as funções de *buzzer* para o da 7, e tivemos êxito. A execução correu como o planejado, e além da versão oferecida em sala, testamos versões alternativas do código pesquisadas na internet, a fim de aprender mais sobre diferentes formas de implementar as mesmas funções com maior eficiência.

6. Bloco de Código:

6.1- Abaixo, está transcrito o código utilizado para a montagem 7, sobre o uso do buzzer implementado sobre o semáforo inteligente.

```
// --- Pinos dos LEDs Veiculares ---
const int PINO_VERDE_V = 10;
const int PINO_AMARELO_V = 11;
const int PINO_VERMELHO_V = 12;
const int PINO_BUZZER = 6;

// --- Pinos dos LEDs de Pedestre ---
const int PINO_VERDE_P = 7;
const int PINO_VERMELHO_P = 8;

// --- Pino do Botão de Pedestre ---
const int PINO_BOTAO_P = 2;

// --- Tempos das Fases (em milissegundos) ---
const unsigned long TEMPO_VERDE_V = 8000;
const unsigned long TEMPO_PISCA_VERDE_V = 2000;
const unsigned long TEMPO_AMARELO_V = 3000;
const unsigned long TEMPO_VERMELHO_V_MIN = 5000;
const unsigned long TEMPO_VERDE_P = 7000;
const unsigned long TEMPO_VERMELHO_P_PISCANTE = 3000;
const unsigned long TEMPO_PISCA_INTERVALO = 300;

// --- Estados do Semáforo (Máquina de Estados) --
-
enum EstadoSemaforo {
    VEICULAR_VERDE,
    VEICULAR_VERDE_PISCANDO,
    VEICULAR_AMARELO,
    VEICULAR_VERMELHO,
    PEDESTRE_PREPARA,
    PEDESTRE_VERDE,
    PEDESTRE_VERMELHO_PISCANDO,
    PEDESTRE_FINALIZA
};

EstadoSemaforo estadoAtual =
VEICULAR_VERDE;
```

```
// --- Variáveis de Controle ---
unsigned long tempoEntradaEstado = 0;
unsigned long ultimoTempoPisca = 0;
bool ledPiscaEstado = LOW;
bool botaoPedestrePressionado = false;
bool pedidoPedestrePendente = false;

// --- Variáveis de Controle do Botão ---
int estadoBotaoEstavel = HIGH;
int ultimoEstadoLeitura = HIGH;
unsigned long ultimoDebounceTempo = 0;
const unsigned long debounceDelay = 50;

void setup() {
    pinMode(PINO_VERDE_V, OUTPUT);
    pinMode(PINO_AMARELO_V, OUTPUT);
    pinMode(PINO_VERMELHO_V, OUTPUT);
    pinMode(PINO_BUZZER, OUTPUT);
    pinMode(PINO_VERDE_P, OUTPUT);
    pinMode(PINO_VERMELHO_P, OUTPUT);
    pinMode(PINO_BOTAO_P, INPUT_PULLUP);

    Serial.begin(9600);
    Serial.println("Semaforo com Chamada de Pedestre Iniciado.");

    mudarParaEstado(VEICULAR_VERDE);
}

void loop() {
    verificarBotaoPedestre();
    controlarSemaforo();
}

void verificarBotaoPedestre() {
    int leituraBotao =
digitalRead(PINO_BOTAO_P);

    if (leituraBotao != ultimoEstadoLeitura) {
        ultimoDebounceTempo = millis();
    }

    if ((millis() - ultimoDebounceTempo) >
debounceDelay) {
        if (leituraBotao != estadoBotaoEstavel) {
            estadoBotaoEstavel = leituraBotao;
```

```

        if (estadoBotaoEstavel == LOW) {
            if (!pedidoPedestrePendente) {
                botaoPedestrePressionado = true;
                pedidoPedestrePendente = true;
                Serial.println("Botao de pedestre
pressionado!");
            }
        }
    }
}

ultimoEstadoLeitura = leituraBotao;
}

void controlarSemaforo() {
    unsigned long tempoNoEstadoAtual = millis() -
tempoEntradaEstado;

    switch (estadoAtual) {

        case VEICULAR_VERDE:
            if (tempoNoEstadoAtual >=
(TEMPO_VERDE_V -
TEMPO_PISCA_VERDE_V)) {
                if (pedidoPedestrePendente) {

mudarParaEstado(VEICULAR_VERDE_PISCAN
DO);
                }
                else if (tempoNoEstadoAtual >=
TEMPO_VERDE_V) {

mudarParaEstado(VEICULAR_VERDE_PISCAN
DO);
                }
            }
            break;

            case VEICULAR_VERDE_PISCANDO:
                piscarLed(PINO_VERDE_V,
TEMPO_PISCA_VERDE_V);

                if (tempoNoEstadoAtual >=
TEMPO_PISCA_VERDE_V) {
                    mudarParaEstado(VEICULAR_AMARELO);
                }
    }
}

```

```

        break;

        case VEICULAR_AMARELO:
            if (tempoNoEstadoAtual >=
TEMPO_AMARELO_V) {

mudarParaEstado(VEICULAR_VERMELHO);
            }
            break;

            case VEICULAR_VERMELHO:
                if (pedidoPedestrePendente &&
tempoNoEstadoAtual >=
TEMPO_VERMELHO_V_MIN / 2) {
                    mudarParaEstado(PEDESTRE_PREPARA);
                }
                else if (!pedidoPedestrePendente &&
tempoNoEstadoAtual >=
TEMPO_VERMELHO_V_MIN) {
                    mudarParaEstado(VEICULAR_VERDE);
                }
                break;

                case PEDESTRE_PREPARA:
                    if (tempoNoEstadoAtual >= 500) {
                        mudarParaEstado(PEDESTRE_VERDE);
                    }
                    break;

                    case PEDESTRE_VERDE:
                        if (tempoNoEstadoAtual >=
TEMPO_VERDE_P) {

mudarParaEstado(PEDESTRE_VERMELHO_PIS
CANDO);
                        }
                        break;

                        case PEDESTRE_VERMELHO_PISCANDO:
                            piscarLed(PINO_VERMELHO_P,
TEMPO_VERMELHO_P_PISCANTE);

                            if (tempoNoEstadoAtual >=
TEMPO_VERMELHO_P_PISCANTE) {
                                mudarParaEstado(PEDESTRE_FINALIZA);
                            }
                            break;
    }
}

```

```

case PEDESTRE_FINALIZA:
    if (tempoNoEstadoAtual >= 1000) {
        pedidoPedestrePendente = false;
        botaoPedestrePressionado = false;
        mudarParaEstado(VEICULAR_VERDE);
    }
    break;
}
}

void mudarParaEstado(EstadoSemaforo
novoEstado) {
    estadoAtual = novoEstado;
    tempoEntradaEstado = millis();
    ledPiscaEstado = LOW;

    apagarTodosLedsVeiculares();
    apagarTodosLedsPedestre();

    Serial.print("Mudando para estado: ");

    switch (estadoAtual) {
        case VEICULAR_VERDE:
            Serial.println("VEICULAR_VERDE");
            digitalWrite(PINO_VERDE_V, HIGH);
            digitalWrite(PINO_VERMELHO_P, HIGH);
            break;
        case VEICULAR_VERDE_PISCANDO:
            Serial.println("VEICULAR_VERDE_PISCANDO");
            digitalWrite(PINO_VERMELHO_P, HIGH);
            break;
        case VEICULAR_AMARELO:
            Serial.println("VEICULAR_AMARELO");
            digitalWrite(PINO_AMARELO_V, HIGH);
            digitalWrite(PINO_VERMELHO_P, HIGH);
            break;
        case VEICULAR_VERMELHO:
            Serial.println("VEICULAR_VERMELHO");
            digitalWrite(PINO_VERMELHO_V, HIGH);
            digitalWrite(PINO_VERMELHO_P, HIGH);
            break;
        case PEDESTRE_PREPARA:
            Serial.println("PEDESTRE_PREPARA");
            digitalWrite(PINO_VERMELHO_V, HIGH);
            digitalWrite(PINO_VERMELHO_P, HIGH);

```

```

        break;
    case PEDESTRE_VERDE:
        Serial.println("PEDESTRE_VERDE");
        digitalWrite(PINO_VERMELHO_V, HIGH);
        digitalWrite(PINO_VERDE_P, HIGH);
        break;
    case PEDESTRE_VERMELHO_PISCANDO:
        Serial.println("PEDESTRE_VERMELHO_PISCANDO");
        digitalWrite(PINO_VERMELHO_V, HIGH);
        digitalWrite(PINO_BUZZER, ledPiscaEstado);
        tone(6, 5274, 3000);
        break;
    case PEDESTRE_FINALIZA:
        Serial.println("PEDESTRE_FINALIZA");
        digitalWrite(PINO_VERMELHO_V, HIGH);
        digitalWrite(PINO_VERMELHO_P, HIGH);
        break;
    }
}

void piscarLed(int pinoLed, unsigned long
duracaoTotalPisca) {
    if ((millis() - ultimoTempoPisca) >
TEMPO_PISCA_INTERVALO) {
        ultimoTempoPisca = millis();
        ledPiscaEstado = !ledPiscaEstado;
        digitalWrite(pinoLed, ledPiscaEstado);
    }
}

void apagarTodosLedsVeiculares() {
    digitalWrite(PINO_VERDE_V, LOW);
    digitalWrite(PINO_AMARELO_V, LOW);
    digitalWrite(PINO_VERMELHO_V, LOW);
}

void apagarTodosLedsPedestre() {
    digitalWrite(PINO_VERDE_P, LOW);
    digitalWrite(PINO_VERMELHO_P, LOW);
}

```

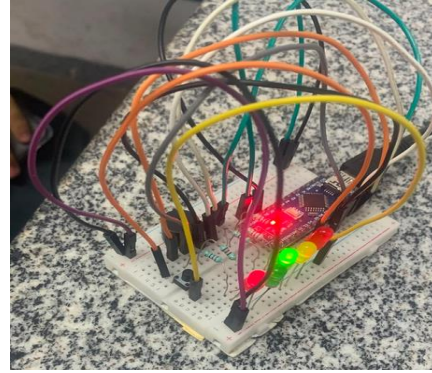
7. Referencias e Imagens:

- Arduino Nano, informações, acesso em: www.arduino.cc
- Arduino IDE, software utilizado.
- Figura do buzzer, acesso em: components101.com/misc/buzzer-pinout-working-datasheet
- Documento da prática 7 acesso no SIGAA.

Arduino NANO, figura 1:



Imagem da montagem concluída, figura 5:



Trecho do código no Arduino IDE, figura 2:

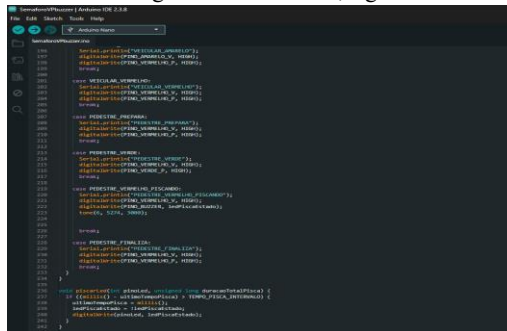


Diagrama do Led utilizado, figura 3:

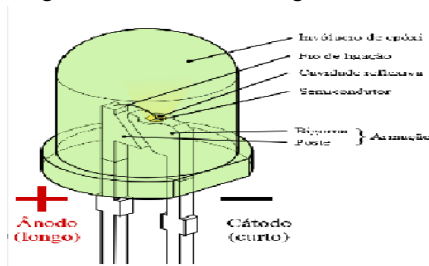


Figura do buzzer digital, figura 4:

